Computers & Security 154 (2025) 104422

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Dynamic anomaly detection using In-band Network Telemetry and GCN for cloud–edge collaborative networks

Jinchuan Pei [a], Yuxiang Hu [a,b,*], Le Tian [a,b], Xinglong Pei [a], Zihao Wang [a]

[a] Information Engineering University, Zhengzhou 450001, China
[b] Key Laboratory of Cyberspace Security, Ministry of Education, Zhengzhou 450001, China

ARTICLE INFO

Keywords:
Anomaly detection
Cloud–edge collaborative network
In-band Network Telemetry
Intrusion detection
Graph Convolutional Network

ABSTRACT

In the intelligent era of the Internet of Everything, the cloud–edge collaborative network architecture solves the data storage and computing problems caused by the exponential growth of terminal data. However, at the same time, the network attack situation is becoming increasingly severe and the types of network anomalies are complex and diverse. The traffic characteristic information collected in traditional network security situation analysis is single and coarse in granularity, which makes it difficult to completely reflect the original traffic and network equipment status. Moreover, the collection of a large amount of fine-grained telemetry data generates substantial telemetry overhead, which hinders the efficient detection of network anomalies and malicious intrusions. To solve this problem, we propose a dynamic anomaly detection method using In-band Network Telemetry (INT) and GCN for cloud–edge collaborative networks, which flexibly and efficiently collects network state information to identify network anomalies and network intrusions. Firstly, we design an anomaly telemetry architecture for cloud–edge collaborative networks and use in-band network telemetry technology of programmable network to extract network characteristic information, and then use dynamic telemetry mechanism to extract network situation elements on demand, so as to quickly identify network anomalies by information entropy method in the edge layer. According to the identified network anomaly information, we deeply telemetry the abnormal position and design a novel Graph Convolutional Network (GCN) that aggregates anomaly information named AGCN in the cloud layer, and analyze whether there is malicious intrusion by combining spatiotemporal dimensions, so that network administrators can accurately grasp the network security situation and discover malicious intrusion in time. The experimental results show that the proposed method can quickly identify network anomalies and detect network intrusions, which can quickly converge while saving telemetry overhead, and the detection accuracy of network intrusions can reach 98.69%.

1. Introduction

With the rapid development of network technologies such as 5G, industrial Internet, and the Internet of Things, the current network scale and traffic information of the whole network are increasing. The emergence of cloud–edge collaborative network architecture solves the data storage and computing problems caused by the exponential growth of terminal data. However, the transformation of the in-network situation under cloud–edge collaborative architecture is complex, and the network security problems such as network failures and malicious attacks are becoming increasingly serious (Li et al., 2023), edge devices are usually deployed in open environments, which are vulnerable to physical attacks and network attacks. Once the edge device is compromised, it may lead to data leakage or system paralysis. Therefore, mastering the network security situation in a complex and changeable network environment is helpful for network administrators to make timely and effective network security protection decisions. Situational awareness refers to extracting elements in the system within a certain time and space, understanding the meaning of elements and predicting possible effects. As an effective means to evaluate network security risks, network security situational awareness plays a vital role in network security defense.

However, the current rapidly changing network situation in the current cloud–edge collaborative network architecture makes it difficult for network administrators to detect network anomalies and malicious traffic in time. Moreover, traditional network security situational awareness methods often ignore the complexity and dynamics of the network, lack fine-grained and accurate network monitoring

* Correspondence to: 7 Jianxue Street, Wenhua Road, Jinshui District, Zhengzhou City, Henan Province, China.
E-mail address: huyuxiangchn@163.com (Y. Hu).

https://doi.org/10.1016/j.cose.2025.104422
Received 25 December 2024; Received in revised form 11 February 2025; Accepted 8 March 2025
Available online 26 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

mechanisms, and cannot accurately and efficiently identify network security threats. Therefore, in the face of the current rapidly changing cloud–edge collaborative network situation, it is urgent to develop a fast and efficient method to perceive network abnormal behavior and identify malicious intrusion.

It should be noted that the terms network anomaly and network intrusion are usually used confusedly (Abdallah et al., 2021). However, in the field of network situational awareness, there are still differences between these two terms: network abnormality usually refers to network behavior deviating from the normal expected pattern, which may be caused by many reasons. Network abnormality is not necessarily caused by network intrusion, including but not limited to software and hardware failures, configuration problems, network congestion, or malicious intrusion attacks; Network intrusion refers to unauthorized malicious behavior, in which attackers try to illegally access, damage, or control network resources, such as DDoS attacks and virus transmission.

To deal with increasingly complex network threats, network administrators not only need to identify network anomalies, but also need to detect whether there are malicious intrusions. The basis of identifying network anomalies and detecting network intrusions lies in collecting network situation elements. Before the appearance of programmable data plane, network administrators could only obtain lagging and inaccurate network telemetry information through terminal devices at the edge of the network, which has poor flexibility and single measurement data type, and it is difficult to accurately and detailed perception of the network security situation. For example, in the current traditional network monitoring tools, the SNMP (Case et al., 1988) polling mechanism is inefficient, with coarse granularity and high delay. Pingmesh (Guo et al., 2015) can provide end-to-end delay measurement information, but cannot directly monitor the device status on the hop-by-hop path. NetFlow (Fernandez et al., 2017) and sFlow (Ujjan et al., 2020) mainly collect stream-level statistical information and cannot provide detailed performance data such as packet transmission delay and packet loss rate.

With the emergence of the P4 language and programmable data plane, In-band Network Telemetry (INT) proposed by P4 organization can collect and monitor real-time network status information in the data plane. With the support of programmable switches, the types of metadata that can be collected become diversified, embedding detailed network performance information when forwarding data packets, and directly obtaining packet-level telemetry data inside the device. Compared with traditional network measurement solutions, INT can achieve end-to-end measurement of network topology, performance, and traffic. INT records the information of each network node in the process of data packet transmission and can obtain fine-grained network status parameters such as switch ID, queuing delay, buffer delay, link delay, incoming/outgoing port ID, incoming/outgoing timestamp, and upload the telemetry information to the controller by the edge switch. This kind of fine-grained data is more sensitive to changes in network status, which helps to accurately locate the source of network problems and anomalies. The switch status parameters obtained by INT technology can reflect the network status in real-time, and detect anomalies in a near-real-time way, thus improving the response speed and detection accuracy. Therefore, using INT technology to monitor the network state can understand the network security situation globally and in detail, and when the network state changes, it can accurately identify whether the network abnormality occurs by extracting and analyzing telemetry data. By using INT to collect network situation information, we can not only obtain more fine-grained temporal characteristics of network state, but also make use of spatial characteristics of network state. Therefore, the application of INT in the field of situational awareness can monitor the changing network state in real-time, flexibly, and fine-grained.

However, existing telemetry schemes lack flexibility in supporting telemetry tasks and adding telemetry metadata. A large amount of fine-grained telemetry data will cause huge telemetry overhead, and when adjacent packets pass through the same switch in a short time interval, the switch state change may be very small. For example, when the rate of data flow is 10Mbps, the time interval between two adjacent packets with a size of 1024 bytes is only 0.8192 ms, while the network situation usually does not change significantly in such a short time. Simply adding all the metadata of telemetry to the data packet not only provides more redundant data for anomaly analysis, but also causes unnecessary network bandwidth and computing resource overhead. After identifying the network anomaly, the network administrator needs to detect whether there is network intrusion according to the anomaly information. To avoid redundant telemetry, we need to make a trade-off between the telemetry overhead and the performance of malicious intrusion identification. Therefore, how to flexibly and efficiently collect network situation information to quickly perceive network anomalies and identify malicious intrusions has become an urgent problem in the field of network security.

Based on the above problems, we propose a dynamic anomaly detection method using In-band Network Telemetry (INT) and GCN for cloud–edge collaborative networks. The main contributions of this paper are summarized as follows:

- We design an anomaly telemetry architecture for cloud–edge collaborative networks, which dynamically adjusts telemetry instructions through the bitmap mechanism, and telemetry network metadata information on demand to mitigate the impact of telemetry on network performance.
- We use the INT of the programmable network to extract the network situation elements in fine granularity, and then quickly identify five kinds of network anomalies through the information entropy method, so as to realize the rapid perception of network anomalies under the premise of ensuring low telemetry overhead.
- Based on the perception of network anomalies, we obtain detailed network state information through the dynamic telemetry mechanism for the depth telemetry of abnormal locations. After that, we construct traffic trajectory graphs and design a novel graph convolutional network that aggregates anomaly information named AGCN, which captures the temporal evolution characteristics of network flows, and analyzes whether there are malicious intrusions in network anomalies by combining spatiotemporal dimensions, so that network managers can efficiently grasp the network situation.

The rest of this paper is organized as follows. Section 2 reviews the related work on INT and its application to anomaly detection, Section 3 gives the network telemetry architecture based on INT, Section 4 introduces INT-driven network anomaly and intrusion awareness method, experiments and analysis are given in Section 5, and finally Section 6 summarizes this paper and looks forward to future work.

2. Related work

The cloud–edge collaborative architecture makes full use of the powerful computing resources of the cloud and the low-latency and efficient performance of the edge, improving the overall efficiency of the system. However, as an emerging technology, it still needs to pay attention to the security issues faced in the application. Qu et al. (2023) proposed a heterogeneous network structure publishing security framework based on cloud–edge collaboration, which solved the problem of imbalance between privacy and availability of network structure privacy protection between heterogeneous nodes in a cloud–edge collaboration network. In order to detect abnormal devices in the cloud–edge collaborative networks, Jiang et al. (2023) proposed an SDN and neural network-based anomaly detection and access control mechanism for cloud–edge collaborative networks, which used reputation-based access restrictions to effectively reduce the number of malicious device attacks.

Most of the research on traditional network anomaly awareness focuses on traffic classification tasks, aiming to improve its detection accuracy, and verifying the effectiveness of the proposed method by using data sets based on network traffic. At present, the research method of network anomaly based on deep learning has become mainstream. Wang et al. (2017) used Convolutional Neural Network (CNN) to classify malicious traffic, but it was easy to fall into the local optimum. Abdallah et al. (2021) proposed a traffic anomaly detection model based on a regularized CNN and Long Short-Term Memory (LSTM) network, which combined CNN and LSTM to learn spatial and temporal features from input traffic, and had better abnormal traffic detection effect than using CNN or LSTM model alone. However, since the original sample categories are unbalanced, it is necessary to manually balance the training data. Liu et al. (2019) proposed a multi-layer encoder–decoder structure that can deeply explore the underlying sequential properties of traffic. Zhao et al. (2022) used a residual graph convolutional network to exploit the relationship between flows for feature extraction. Hnamte and Hussain (2023) built a network traffic classifier based on a deep CNN optimization model in the SDN network environment, which has a high detection accuracy.

However, in the cloud–edge cooperative network, the data characteristics of the edge layer change greatly, and it requires rapid response to the real-time changes of the data. The above-mentioned research that only uses deep learning technology requires a large number of training parameters, has low detection efficiency, and fails to distinguish network anomalies in a fine grain. It relies on label data to obtain effective features, and cannot provide support for network administrators to detect malicious intrusions in real-time when there is newly incoming traffic data in the network.

To perceive the network security situation in real-time, in recent years, the industry has begun to use network telemetry technology to research network real-time monitoring. The network status can be monitored by mapping packet mirrors on the switch (Handigol et al., 2014; Rasley et al., 2014; Tilmans et al., 2018; Zhu et al., 2015), but the collected data cannot judge the global network status, and it is easy to cause the omission of abnormal event diagnosis. Liu et al. (2024) designed R-Pingmesh, a service-aware RoCE network monitoring and diagnosis system, to collect probe packets to detect and diagnose network faults. The NetBouncer proposed by Tan et al. (2019) actively probes paths in data center networks and can detect failures of devices and links. However, NetBouncer uses a detection interval of minute-level failures, which will filter out micro-burst anomalies. These researches can realize the diagnosis of network state, but the probe packets measured out of band cannot cover all forwarding paths and flow entries, and it is easy to cause bandwidth waste.

At present, INT has been applied to network status monitoring. Singh et al. (2019) concluded through their research that INT has the advantages of low latency and high throughput for monitoring network status, and can meet the requirements of metadata acquisition in network situation awareness.

Yao et al. (2018) designed the NetworkAI architecture in SDN. They adopt INT to obtain millisecond fine-grained network telemetry data, when the packet is sent to INT Sink, load data will be sent to the user, telemetry data will be sent to the network analysis plane and decision plane, and finally intelligent data collection and network status analysis are implemented based on actual traffic.

To implement the deployment of network monitoring tasks, Yang et al. (2020) proposed a lightweight and efficient INT framework named Fast-INT. Fast-INT converts network telemetry into reinforcement learning tasks, marks INT metadata as reinforcement learning states, sets actions to record high-load links, and calculates rewards according to the network state after executing actions. Thus, it uses the INT scheduling algorithm based on reinforcement learning to realize the deployment of network monitoring tasks.

Some studies have applied INT telemetry data to network anomaly detection. Zhang et al. (2021) proposed INT-Detector, a network anomaly detection system combining INT and deep learning, and used generative adversarial active learning to detect anomalies, aiming to use low-pass filters to improve accuracy in the case of network traffic fluctuations. Cao et al. (2021) sense the status of switches by INT and collect network status information by adaptive sampling. Then it detects whether the switch is attacked based on the switch status information, but it cannot quickly identify network anomalies, and the training process of detecting attacks requires high expert knowledge. Zhan et al. (2023) made use of the flexible control ability of INT and programmable network, and applied deep reinforcement learning to analyze switch entrance and exit information to locate link anomalies. However, they only analyzed the network anomaly of delay. Sacco et al. (2023) proposed an INT architecture named HINT, which instructs switches to embed simple device information such as processing delay and queue usage into transmitted packets, thus providing decision control for network congestion. Duan et al. (2023) used a deep learning method based on the graph attention mechanism to preprocess telemetry data, and then made abnormal judgments and alarms according to the threshold. However, when the network state fluctuates frequently, the dynamic of threshold update is not good, which makes it easy to cause false alarms in subsequent detection.

Subsequently, Wang et al. (2023) designed a MARS system to perform dynamic threshold anomaly detection and locate the fault cause.MARS collects packet-level telemetry data according to requirements in the programmable network system, but it only sets high delay and packet loss event detection. Its extended fault cause types require the intervention of expert knowledge, which requires experts to analyze and summarize the fault causes, and design corresponding features for fault matching.

However, with the expansion of network scale and the increase of high-speed traffic, using INT to collect the whole network situation information will cause huge network telemetry overhead. Zhao et al. (2021a) proposed a threat situation awareness method based on the Internet of Things (IoT) edge and network traffic for IoT architecture to reduce the dimension of threat traffic, summarize it and generate situation information. However, the objects of feature extraction and traffic cleaning are all network edge nodes, and the existing network anomalies in the switching nodes cannot be found.

In summary, in the research of security situation awareness of cloud–edge collaborative network, traditional anomaly detection methods are difficult to completely reflect the original traffic and network equipment status, and cannot effectively identify network anomalies and warn of intrusion behaviors. It is difficult to balance the anomaly detection performance and network telemetry overhead when applying INT to network anomaly detection. Therefore, how to better apply deep learning-based anomaly detection algorithms to network anomaly detection with INT data to achieve efficient and accurate network anomaly awareness still needs further research and exploration.

3. Cloud–edge collaborative network telemetry architecture

Fine-grained monitoring in cloud–edge collaborative environment is the prerequisite for realizing high-precision network anomaly awareness. In this section, we design the cloud–edge collaborative telemetry architecture for network anomaly awareness.

3.1. Overall telemetry framework

The overall telemetry framework of the cloud–edge collaborative network is shown in Fig. 1. The telemetry system we designed includes two parts: the cloud layer and edge layer.

The edge layer uses INT technology to collect and report the current network status, collect the device-level telemetry data that can be used as input for anomaly detection model training in real-time, and send the important telemetry information to the cloud layer storage. Edge layer devices include INT source, INT transit, INT sink, and edge layer

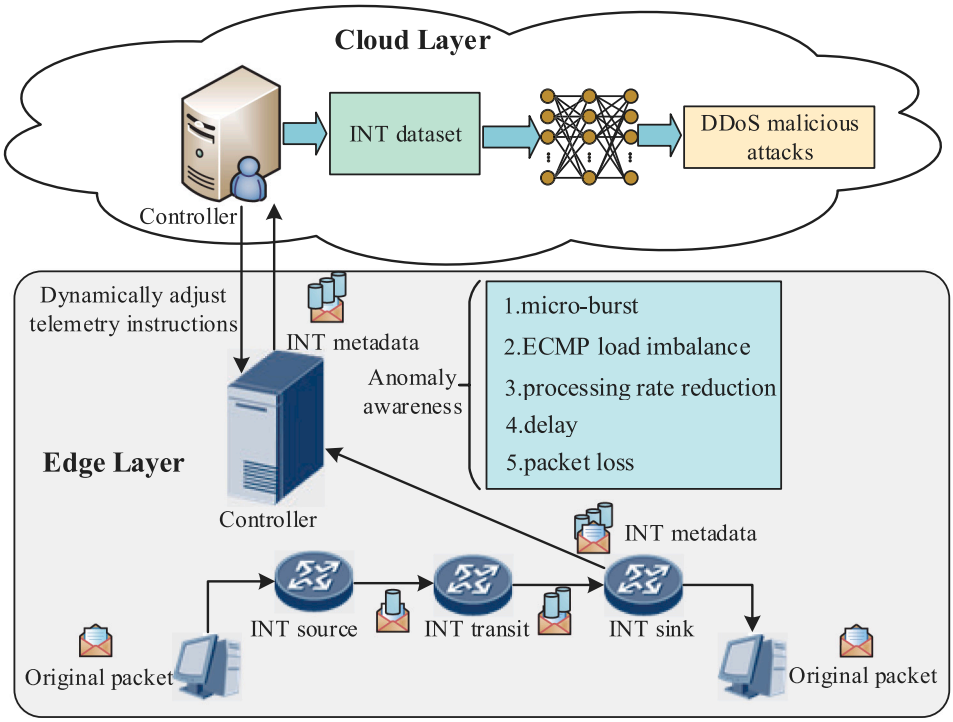

Fig. 1. Overall telemetry architecture.

controller. At the INT source switch, the telemetry system periodically samples the network packets. INT technology can obtain the fine-grained parameters of network traffic and devices, embed telemetry instructions into the header of telemetry data packets, and then collect the internal state information of network devices at each hop on the path, such as timestamp and queue length. In the subsequent routing and forwarding process, the intermediate forwarding device interprets the instructions and embeds the relevant data, that is, every time the data packet passes through an INT transit hop, it inserts a new INT metadata according to the telemetry instructions. When the switch reaches the last switch with telemetry function (INT sink), the INT sink inserts own state information and extracts the telemetry information, which is uploaded to the edge layer controller. The whole process collects and presents the network state according to the real-time forwarding and measurement process, without knowing the complete network topology and traffic transmission path in advance.

In the edge layer of Fig. 1, after the terminal sends the data packet to the INT source switch, the INT source switch collects the internal state of the device (INT metadata 1) and inserts it into the INT metadata stack. After the packet is forwarded to the INT transit switch, the header is parsed and the device status (INT metadata 2) is collected, which is continued to be pushed into the INT metadata stack. The last hop of the telemetry path (INT sink) inserts the last device state information (INT metadata 3) into the metadata stack and sends the extracted telemetry information to the edge layer controller for further processing to quickly sense whether there is a network anomaly.

The cloud layer is responsible for the telemetry information configuration, telemetry instruction delivery, monitoring frequency setting, etc., and collects key network status information telemetry from the edge layer to build an INT data set as input training/processing data to send to the intrusion detection model. When the edge controller received the network telemetry information and used the information entropy-based method to quickly perceive whether there was a network anomaly, the cloud layer continuously maintained the storage structure of the edge controller to dynamically update the information entropy threshold. However, in today's high-speed network, identifying network anomalies requires a large amount of network state information. The edge layer devices are limited by computing and storage resources, and it is difficult to monitor the state information of a large amount of traffic at the same time. Extracting all the detailed telemetry information when the packet flows through the switch will generate a lot of network overhead. Therefore, in the case of fast-detecting network anomalies, we only telemetry the key traffic characteristics that can be used for anomaly awareness through the dynamic telemetry mechanism.

Once a network anomaly is detected, it locates the data flow or switch node where the anomaly occurs according to the telemetry information, and dynamically adjusts the telemetry instructions, so that the edge layer can further telemetry the abnormal location to focus on monitoring the abnormal location and the telemetry data type that needs to be concerned. Then, the cloud layer detects whether DDoS attacks exist in the network through the AGCN that aggregates abnormal information, so that the network administrator can master the network security situation.

3.2. Dynamic telemetry mechanism

The maximum Transmission Unit (MTU) of a packet in the network is 1500 bytes. If the telemetry metadata is blindly added to the packet, the packet length exceeding the MTU will not be transmitted. Therefore, it is very important to deploy network telemetry in a targeted manner. Telemetry should balance the granularity of state information and resource consumption such as bandwidth storage.

When the network anomaly is found, the network administrator still needs to further determine whether there is a malicious intrusion. Therefore, when the network anomaly is identified, the abnormal location and traffic are further telemetry, and the network telemetry information is dynamically collected.

In dynamic telemetry, we designed an INT field to flexibly control the data type of INT telemetry, that is, to send telemetry instructions through dynamic telemetry and select telemetry information that needs further attention. Adding metadata flexibly according to the telemetry task can minimize the network telemetry overhead, and at the same

Table 1
Example of the bitmap mechanism.

Bitmap	Metadata
1	Switch id
. 1	Port id
. . 1	Ingress timestamp
. . . 1	Egress timestamp
. . . . 1	Hop latency
. . . . . 1	Queue depth
. . . . . 1	Link speed
. . . . . 1	Queue occupancy
. . . . . 1	–

time, it can avoid modifying and updating the data plane source code due to the change of telemetry task.

We use the bitmap mechanism to define telemetry tasks in the stage of rapid detection of network anomalies, and dynamically select metadata for measurement according to telemetry tasks, so that switches can flexibly add telemetry metadata and analyze whether there is a network anomaly according to the current network state. In the INT header, we set the bitmap field, where each bit corresponds to a telemetry metadata type. As the packet passes through the switch processing pipeline, match the INT header field and check for all bits set to 1 in the bitmap field. The INT metadata of the corresponding type is then embedded in the INT packet as a new metadata field. The types of metadata that can be telemetry in INT are diverse, and the metadata information that can be telemetry can be queried in technical documents (Changhoon et al., 2020). Network administrators need to selectively measure the metadata information according to the network status. For example, as shown in the bitmap field in Table 1, a 16-bit bitmap indicates that network telemetry can control the collection of 16 types of metadata. For example, if a telemetry task for micro-burst detection requires the switch to provide metadata such as switch ID, timestamp, and queue depth, it can be set to bitmap = 0b1011010000000000.

4. Anomaly-based intrusion detection

To facilitate network administrators to grasp the network security situation, this section proposes an INT-driven anomaly-based intrusion detection method. The proposed method is divided into two stages: network anomaly detection based on information entropy and malicious intrusion identification based on aggregation of abnormal information. Firstly, in the edge layer, the information entropy method is used to quickly find network anomalies and preliminarily identify abnormal terminals and traffic. Based on sensing network anomalies, the telemetry instructions are dynamically adjusted, and the detailed network state information of abnormal locations is obtained by deep telemetry through the dynamic telemetry mechanism. After that, the cloud layer analyzes whether there is a malicious intrusion in the network anomaly through the graph convolutional network AGCN that aggregates abnormal information. If there is a malicious intrusion, the situation warning level of the cloud–edge collaborative system is improved.

4.1. Telemetry model

In this section, the network feature elements telemetered at the edge layer are mapped to the network model. INT information can be divided into switch level and data flow level. Each switch node in the network can generate multiple groups of telemetry data corresponding to different data flows. To build a unified view of telemetry, it is first necessary to aggregate the telemetry data, which can be divided into horizontal aggregation and vertical aggregation. Horizontal aggregation is the aggregation of metadata reported by switches on all paths that a packet passes through. Vertical aggregation is to arrange the metadata reported by a switch in order of time for the analysis of switch timing data.

We integrate the features of switch-level telemetry and flow-level telemetry horizontally and vertically, that is, for each flow, we integrate the telemetry of each switch it passes through to create a comprehensive network situation telemetry record.

During the telemetry process, the packet information flowing through each switch is recorded. After the timestamps of the telemetry data are aligned, the chronological order in which the data flows through each switch can be analyzed.

The telemetry information collected by all switches through which each data flow flows is regarded as a row vector. For example, when the data flow f_i flows through switch s_j, its state information is represented as a telemetry feature vector $a_{ij} = [t, d, p]$, including timestamp, queue depth, and processing delay, etc. For each switch s_j, all the data flow information flowing through it can be represented as a telemetry aggregation matrix $SA_j = \left[a_{1j}, a_{2j}, \ldots, a_{mj}\right]^{\mathrm{T}}$. If there are n switch nodes in the network, the INT information can be represented as a matrix $M = \left[SA_1; SA_2, \ldots, SA_n\right]$, where each row is the telemetry of the data flow f_i through each switch, and each column represents a telemetry feature. For example, there are two data flows f_1 and f_2 that go through switches (s_1, s_2) and (s_1, s_3), respectively. Each switch collects three parameters: timestamp t, processing delay d, and queue depth q. Then the telemetry matrix M will have two rows (one row for each data flow) and nine columns (the sum of telemetry parameters of switches passed by each data flow), which can be denoted as:

$$M = \begin{bmatrix} t_{s_1,f_1} & d_{s_1,f_1} & q_{s_1,f_1} & t_{s_2,f_1} & d_{s_2,f_1} & q_{s_2,f_1} & 0 & 0 & 0 \\ t_{s_1,f_2} & d_{s_1,f_2} & q_{s_1,f_2} & 0 & 0 & 0 & t_{s_3,f_2} & d_{s_3,f_2} & q_{s_3,f_2} \end{bmatrix}. \tag{1}$$

For convenience, we summarize notations that appear in this paper in Table 2.

4.2. Network anomaly detection

Network anomaly detection can be used to identify the abnormal behavior that may occur in security incidents. When the traffic transmission behavior in the network deviates from the normal expected pattern, it can be identified as an anomaly. In this section we classify network anomalies into five categories, namely micro-burst, Equal-Cost Multi-Path (ECMP) load imbalance, processing rate reduction, delay and packet loss.

Due to the size limit of MTU of a telemetry packet in INT, it will cause invalid communication overhead if a large number of data are sampled in each telemetry. Therefore, when performing network anomaly detection, key information such as timestamp and queue occupancy can be given priority, and the telemetry information can be quickly detected according to the method based on information entropy. Information entropy is a quantitative index to measure the uncertainty of information. For network anomaly detection, using information entropy can help us quickly identify the patterns that deviate from normal network behavior.

We obtain the switch state and traffic characteristic information for anomaly detection through INT, and then calculate the entropy value of the characteristic information to obtain the dispersion degree of each traffic characteristic information, and compare it with the dynamic threshold to determine whether it is abnormal traffic. We comprehensively analyze multiple metadata from telemetry to detect whether there are network anomalies. The following is the method of detecting five anomalies using INT data based on information entropy.

4.2.1. Micro-burst

Micro-burst refers to the influx of a large number of packets in a short period in the network, resulting in strong traffic spikes that can lead to buffer overflow and packet loss. During a micro-burst, the queue length increases rapidly, and INT can record the precise timestamp and

Table 2
Summary of notations.

Notation	Description
f_i	The ith data flow
s_j	The jth switch
a_{ij}	Feature vector of telemetry information produced by data flow f_i flowing through switch s_j
SA_j	Telemetry aggregation matrix of the switch s_j
t	Time stamp
d	Processing latency
q	Queue depth
T	Sequence of packet interarrival time intervals
Ts	Dynamic threshold value
μ_H	Average of historical information entropy
σ_H	Standard deviation of historical information entropy
$R = \{r_1, r_2, \dots, r_K\}$	ECMP path group in the network
N_k	Total number of packets actually passing through path during telemetry
K	Total number of paths in the ECMP group
I_k	ECMP Imbalance index
$E_{s(t)}$	Packet processing capability of switch s per unit time
$N_{\Delta t}$	Total size of packets passing through the switching node s per unit time
$\Delta Q_{s(t)}$	Amount of change in the queue length of the switching node s in unit time
l_s	Latency per hop
$X = \{x_1, x_2, \dots, x_n\}$	Network anomaly label
$Y = \{y_1, y_2, \dots, y_n\}$	Malicious intrusion label
$G = (V, E)$	Structure of traffic trajectory graph
$M \in \mathbb{R}^{n \times m}$	Characteristic information matrix of nodes in the traffic trajectory graph
$A \in \mathbb{R}^{n \times n}$	Edge connection matrix in the traffic trajectory graph
$D \in \mathbb{R}^{n \times n}$	Degree matrix of the trajectory graph
W	Weight matrix used for the convolution operation
M_{anom}	Feature matrix that records network anomaly information

switch queue length of each packet as it passes through the switch node, which can detect the micro-burst.

According to the timestamp of the packet arrival at the switch, the packet arrival time interval sequence $T = \{t_1, t_2, \dots, t_n\}$ is recorded, where t_i is the arrival time interval between the i th packet and the previous packet. Meanwhile, the queue length $Q = \{q_1, q_2, \dots, q_n\}$ of the current time is recorded, and the joint information entropy is set as:

$$H(T, Q) = -\sum_{i=1}^{n} P(t_i, q_i) log_2^{p(t_i, q_i)}, \tag{2}$$

where $P(t_i, q_i)$ is the statistical probability of the queue length change in the packet arrival time interval, which is calculated by frequency statistics based on historical data. By monitoring the change of joint information entropy, when the information entropy suddenly increases and exceeds the dynamic threshold, it indicates that a micro-burst occurs. The data obtained from the data plane is stored in memory to dynamically update the historical information entropy. According to the statistical characteristics of the historical information entropy, the dynamic threshold Ts is set as the average information entropy plus k times the standard deviation:

$$Ts = \mu_H + k\sigma_H. \tag{3}$$

Among them, μ_H is the average of the historical information entropy, σ_H is the standard deviation of historical information entropy, and the k value is greater than 1, to ensure that the threshold is sensitive enough. If the calculated information entropy $H(T, Q)$ exceeds the threshold Ts in a certain time window, it is considered that a micro-burst occurs in the time window. It should be noted that in order to prevent the influence of network outliers on the historical information entropy, when network anomalies are detected, the telemetry information obtained in this stage is not included in the statistics of historical information.

4.2.2. ECMP load imbalance

As a routing policy, ECMP is used to distribute traffic over multiple paths to balance the load. Network load imbalance may occur when routes are improperly configured or under malicious DDoS attacks. By collecting the switch queue cache information over a period of time with INT, the traffic distribution on different paths can be analyzed and the traffic imbalance can be identified.

INT By collecting the switch ID and timestamp that the packet flows through, we can determine the traffic transmission path as $R = \{r_1, r_2, \dots, r_K\}$, where each r_k represents an independent path and K is the total number of paths in the ECMP group. We define the ECMP imbalance index I_k to quantify the degree of load imbalance of path r_k, which can be formulated as:

$$I_k = \left| \frac{N_k}{\sum_{k=1}^{K} N_k} - \frac{1}{K} \right|, \tag{4}$$

where N_k is the total number of packets passing through path r_k during telemetry, while $\Sigma_{k=1}^{K} N_k$ is the total number of packets for all paths in the ECMP group. In the ideal case, all paths are perfectly load balanced, that is, each path r_k equally carries $\frac{1}{K}$ of the total traffic. The imbalance index I_k can be obtained by taking the difference between the actual and ideal load ratio. The higher the difference, the more the load of path r_k deviates from the average.

We use information entropy to quantify the overall degree of load imbalance across all paths, and we normalize the imbalance index to be a probability distribution. The normalized probability can be defined as follows.

$$p(I_k) = \frac{I_k}{\sum_{k=1}^{K} I_k}. \tag{5}$$

$\sum_{k=1}^{K} I_k$ is the sum of all imbalanced exponents, ensuring that the sum of all $p(I_k)$ is equal to 1. After obtaining the normalized probability, the entropy $H(I)$ of the load imbalance index is calculated by using the information entropy formula:

$$H(I) = -\sum_{k=1}^{K} p(I_k) log_2^{p(I_k)}. \tag{6}$$

If $H(I)$ is higher than the dynamic threshold Ts, the distribution of the load imbalance index is scattered, that is, there is a serious load imbalance.

4.2.3. Processing rate reduction

The reduction of processing rate may be caused by the limited resources such as CPU and memory of the switch or malicious attacks.

INT can determine the packet processing rate of each switch by recording the packet size, the incoming and outgoing timestamp, and the queue length of the switch. If the processing time of a certain network device increases significantly, it indicates that there is a processing rate degradation. $E_{x(t)}$ is the packet processing capacity of switch s in unit time. Considering the impact of queue length on processing capacity, the processing rate of switch $E_{s(t)}$ is formulated as:

$$E_{s(t)} = \frac{N_{\Delta t} v}{v\Delta t + \Delta Q_{s(t)}}, \tag{7}$$

where $N_{\Delta t}$ is the total size of data packets passing through switch node s in unit time, $\Delta Q_{s(t)}$ is the change in the queue length of switch node s in unit time, weighting factor v is used to adjust the impact of queue length on processing efficiency, and is set to the rated normal rate of switch processing packets. Once the processing rate $E_{s(t)}$ is obtained, the frequency is calculated and normalized, and then the information entropy $H(E)$ is used to evaluate the distribution stability of the processing rate.

4.2.4. Delay

Network delay can be caused by a variety of reasons, such as hardware and software failures, network congestion, or traffic hijacking. With INT, we record the arrival timestamp of each hop to measure the end-to-end latency and the latency per hop, helping to locate the specific locations where the network transmission delay increases.

Each device records the ingress and egress timestamps of the packets it processes. For the switching node s, we have an ingress timestamp $t_{s(in)}$ and an egress timestamp $t_{s(out)}$, and the latency per hop can be calculated by the following formula:

$$l_s = t_{s(out)} - t_{s(in)}, \tag{8}$$

After that, the delay distribution was constructed and the delay distribution probability was calculated. Finally, we calculate the information entropy value. If the delay entropy value exceeds its dynamic threshold, it is considered that the network delay is abnormal.

4.2.5. Packet loss

Packet loss is usually caused by network congestion, device failure, or malicious intrusion. The INT technique is used to trace the whole path of the packet, including the switch ID and the packet count. If the number of packets decreases after a certain switch node, the packet loss event can be inferred.

4.3. Algorithm design

After clarifying the above five types of network anomalies based on telemetry information such as switch status, we define the anomaly label $X = \{x_1, x_2, \ldots, x_n\}$, $x_i \in \{0, 1\}$ denotes whether there is a network anomaly in the telemetry data information of the switch s_i, 1 means abnormal, 0 means normal, which is used to locate the anomaly and provide abnormal information for malicious intrusion recognition.

The five network anomalies can be mapped to two fault categories, micro-burst anomalies can be mapped to the corresponding traffic, ECMP load imbalance, processing rate reduction, delay, and packet loss can be mapped to the corresponding switch. Abnormal data flows and abnormal nodes can be determined according to the fault categories mapped by network anomalies. Then, the telemetry task is dynamically adjusted according to the abnormal state of the network, the abnormal traffic and switching nodes are telemetry deeply, and the AGCN that aggregates abnormal information is used to analyze the telemetry data to further determine whether there is a malicious intrusion in the network anomaly. The overall process is shown in Algorithm 1.

Firstly, when the source node receives the data packet, it starts to issue telemetry instructions and collect network telemetry information, initializing the anomaly label and intrusion label as 0 (line 1–5). Then the loop determines whether there are network anomalies, and the anomaly_type is five kinds of network anomalies in turn. After analyzing the telemetry data, the information entropy H and the dynamic threshold Ts are calculated respectively. If the value is lower than the dynamic threshold, there is no network anomaly and the threshold is updated (line 6–10). Otherwise, the anomaly label is updated, the telemetry instruction is dynamically modified and the transmission path r_i with anomaly is further telemetry to update the INT information matrix M. The information matrix M already contains the abnormal feature information (line 11–16). Finally, the AGCN with aggregated abnormal information was used to deeply identify whether there was malicious intrusion (line 17–21). Therefore, it can not only detect whether there is an anomaly in the network, but also further judge whether there is a malicious intrusion according to the anomaly, and locate network anomalies by outputting anomaly label X and intrusion label Y (line 22–23), so that the network administrator can master the network security situation.

Algorithm 1 INT-driven network anomaly awareness

Input: Telemetry instructions
Output: Anomaly label X, malicious intrusion label Y

```
1:  while receive flow f_i do
2:      Set the bitmap field to send telemetry instructions;
3:      calculate INT Information matrix M;
4:      x_i = 0;
5:      y_i = 0;
6:      for each anomaly_type in Network Anomaly do
7:          Calculate H and Ts;
8:          if H < Ts then
9:              update Ts;
10:             break;
11:         else
12:             x_i = 1;
13:             reset the bitmap field for r_i;
14:             update INT Information matrix M;
15:         end if
16:     end for
17:     if x_i = 1 then
18:         if AGCN(M) is malicious intrusion then
19:             y_i = 1;
20:         end if
21:     end if
22:     return X,Y;
23: end while
```

4.4. Malicious intrusion detection

In order for administrators to grasp whether the network is under malicious intrusion, on the basis of quickly detecting network anomalies, it is also necessary to analyze the causes of anomalies. For example, processing rate reduction and network delay anomalies may be caused by network congestion, software and hardware failures, or security attacks. It is necessary to further determine whether it is subjected to security attacks such as DDoS attacks.

The AGCN malicious intrusion detection method of aggregating abnormal information proposed in this subsection detects malicious intrusions in both spatiotemporal dimensions. In the temporal dimension, the network anomalies are concatenated by the timestamp of the packets forwarded by the switch where the anomalies are located, and the anomaly correlation information is captured by the network topology coding in the spatial dimension.

4.4.1. Overview of GCN

Graph convolutional networks (Zhang et al., 2019) are neural networks that operate directly on non-Euclidean distances and can multiply graph convolutional layers with spectral filters in the Fourier

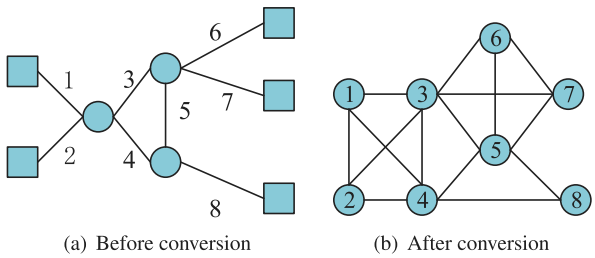

Fig. 2. Example of a traffic trajectory graph.

domain to aggregate node information. By stacking graph convolution layers, the feature information in switch nodes spreads among different layers, which helps to discover the relationship between different network anomalies, accurately judge and identify malicious network intrusion, and helps network administrators grasp the network security situation.

In intrusion detection, the interpretation of the weight of each node or edge is helpful in understanding which nodes or edges have more important contributions to the detection. GCN uses graph structure information in the training process. Compared with ordinary graph neural network, GCN applied to malicious intrusion detection can effectively capture the relationship between nodes and edges, and understand the network topology and evolution process as a whole.

In this paper, INT data is used for malicious intrusion detection. Since network traffic data has complex graph structure characteristics, such as the association between traffic, GCN can better model this graph structure, and the cloud layer in the cloud–edge collaborative architecture has powerful computing resources. Therefore, using the improved GCN method in the cloud layer to process INT information and identify malicious traffic intrusion can improve detection accuracy and performance.

4.4.2. Construct the traffic trajectory graph

Each switch node that the data packet passes through has its characteristic information in the telemetry process. These characteristics are the original network state attribute values, which can be updated and learned in the process of training, helping the model to learn more correlation information. In order to enable GCN to learn the spatial relationship of traffic, we constructed a traffic trajectory graph, extracted the switch nodes and their state information in the INT information to identify the network topology, and used the timestamp to judge the sequence of traffic through the device to generate the traffic transmission path. After comprehensively considering the temporal and spatial characteristics of the network transmission traffic, the relationship between the network traffic is formed into the connection edge to establish the graph model of the traffic, so as to generate the traffic trajectory graph.

An example of constructing a traffic trajectory graph is shown in Fig. 2. The connection edge in Fig. 2(a) represents the traffic transmitted in the network. According to the spatial connection relationship of the transmission traffic in the network, the data flow is regarded as the node of the graph structure, and is transformed into the traffic trajectory graph as shown in Fig. 2(b). For example, in Fig. 2(a), traffic path 1 is connected to path 2, path 3, and path 4, so after converting into the traffic trajectory graph shown in Fig. 2(b), node 1 is directly connected to node 2, node 3 and node 4. Therefore, if the traffic relationship transmitted in the network is regarded as a graph, we can obtain the connection of all the nodes in the graph and their characteristics.

$G = (V, E)$ represents the structure of the traffic trajectory graph, where E represents the set of edges in the traffic trajectory graph, $V = \{v_1, v_2, \dots, v_n\}$ represents the set of nodes in the traffic trajectory graph, n is the number of nodes in the traffic trajectory graph, and the matrix $X \in \mathbb{R}^{n\times m}$ is used to store the characteristic information of nodes in the traffic trajectory graph. Each node $v_i \in V$ in the traffic trajectory graph has its corresponding characteristic information $X_i = \{x_{i1}, x_{i2}, \dots, x_{im}\}$, and the feature dimension of this node is at most m.

To implement the graph convolution operation, we define node correlation in the node domain, and the adjacency matrix $A \in \mathbb{R}^{n\times n}$ of the graph represents the edge connection relationship in the traffic trajectory graph, such as $A_{ij} = 1$ indicates that there is an edge connected between node v_i and node v_j, that is

$$\begin{cases} A_{ij} = 1, \text{if } e_{ij} \in E \\ A_{ij} = 0, \text{if } e_{ij} \notin E. \end{cases} \tag{9}$$

The degree matrix $D \in \mathbb{R}^{n\times n}$ of the trajectory graph is a diagonal matrix, where D_{ii} represents the number of nodes directly connected to node v_i. The diagonal value is calculated as follows:

$$D_{ii} = \sum_{j\in V} A_{ij}. \tag{10}$$

After the construction of the traffic trajectory graph, GCN can be applied to the traffic trajectory graph to capture more detailed network traffic information. Therefore, the problem of malicious intrusion detection is transformed into the classification problem of nodes in the traffic relationship graph, and the spatio-temporal characteristics of traffic are effectively used to analyze the network security situation.

4.4.3. AGCN

INT technology realizes fine-grained measurement of network status, and further identifies whether there is a network intrusion according to the abnormal information after finding network anomalies. Traditional malicious intrusion detection relies on the statistical characteristics of traffic, but ignores the topological spatial relationship between traffic. This method uses the AGCN method to aggregate abnormal information to analyze the switch status characteristics and other information, so as to grasp the spatial relationship of network traffic, and combines the detected network abnormal characteristics to identify malicious intrusions.

GCN uses the global traffic trace node features for classification, so as to detect malicious intrusions. Each traffic trajectory node in the traffic trajectory graph has its corresponding feature matrix. In addition, the feature information is obtained from neighbor nodes and fused with the feature information of its node for analysis. Eq. (11) gives the calculation formula of graph convolution:

$$M_{basic}^{(l+1)} = ReLU\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}M_{basic}^{(l)}W^{(l)}\right), \tag{11}$$

where $M_{basic}^{(l)}$ represents the matrix composed of network state information collected by INT, $W^{(l)}$ represents the weight matrix of the lth layer convolution, $ReLU$ is the activation function, and $M_{basic}^{(l+1)}$ represents the feature matrix of the node in the $(l+1)$th layer after convolution, $\tilde{D}$ is the degree matrix computed based on $\tilde{A}$.

In order to make each node consider its characteristic information when aggregating the information of the associated traffic, Eq. (12) adds an identity matrix I to the original adjacency matrix A, which is equivalent to adding a self-loop for each traffic trajectory node in the graph.

$$\tilde{A} = A + I. \tag{12}$$

After network anomalies are detected, the anomalies can be located. In order to make full use of the important feature information of network anomaly in malicious intrusion detection, we design AGCN to aggregate anomaly information, and the anomaly information detected in network anomaly detection is regarded as an independent feature matrix. If an anomaly is detected in a certain switch, the abnormal information will be recorded through the abnormal feature matrix M_{anom}. The convolution operation of the abnormal feature is as follows:

$$M_{anom}^{(l+1)} = ReLU\left(M_{anom}^{(l)}W_{anom}^{(l)}\right), \tag{13}$$

where $W_{anom}^{(l)}$ is a weight matrix dedicated to network anomaly features, which is used to learn to extract features from network anomaly information and integrate them into the node representation. Independent processing of network anomaly features can make the model more accurate in identifying malicious intrusions.

In order to aggregate abnormal information for convolution operation, we define two weight coefficients α and β to aggregate abnormal features and telemetry information, where $\alpha+\beta = 1$. The feature matrix $M^{(l)}$ of aggregated abnormal information can be obtained by adjusting the weight coefficients.

$$M^{(l)} = \alpha M_{basic}^{(l)} + \beta M_{anom}^{(l)}. \tag{14}$$

After aggregating the abnormal information, the model can focus more on the learning of network abnormal features while maintaining the integrity of telemetry information features and the basic structure of the GCN aggregation step. Note that the weight coefficients and anomaly weight matrix need to be tuned during training to find the best feature weights and ensure that the key information of network anomaly features is not diluted by other telemetry information.

By stacking multiple GCN layers and aggregating the features of neighbor nodes to update the feature representation of nodes, the model can capture deeper feature correlation information in the graph structure. We build a two-layer GCN network, so that the network telemetry information is propagated among the nodes of the traffic trace graph, and the feature representation of the nodes are updated in each iteration step. The historical telemetry information and abnormal information are effectively used for malicious intrusion classification, so as to improve the accuracy and practicality of intrusion detection.

In order to facilitate classification, after a two-layer graph convolution operation, we input the output features into a fully connected layer to further extract and integrate feature information. The fully connected layer can map the node features output by the GCN layer to a smaller feature space, and its output can be expressed as

$$Z = M^{(l)} W^{(fc)} + b, \tag{15}$$

where $W^{(fc)}$ is the weight matrix of the fully connected layer and b is the bias term. The topological relationship and statistical characteristics of traffic can be effectively learned by stacking the two graph convolutional layers, and the network abnormal feature information can effectively enhance the accuracy and robustness of detecting malicious intrusion.

The identification of malicious intrusion can be regarded as a classification problem, and based on the output of the fully connected layer, we can judge which category the nodes in the traffic trace graph belong to, malicious or normal. The sigmoid function is applied in the last layer to obtain the probability of malicious traffic, as shown in Eq. (16):

$$\hat{y}_i = \sigma(z_i), \tag{16}$$

where z_i is the ith element of the output vector Z of the fully connected layer, and its probability value is selected as the final classification result. For classification problems, the AGCN model uses binary cross-entropy as the loss function, as shown in Eq. (17):

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln(\hat{y}_i) + (1-y_i)\ln(1-\hat{y}_i)\right]. \tag{17}$$

N is the total number of samples, y_i is the true label of the ith sample, the value is 0 or 1, and $\hat{y}_i$ is the class probability of the ith sample predicted by the model, when $y_i = 1$, it means that the ith sample is malicious traffic. When $y_i = 0$, it means that the ith sample is normal traffic. By calculating the loss function and using the Adam optimization algorithm, the bias term and weight parameters can be updated. The Adam algorithm can calculate an adaptive learning rate for different parameters to minimize the loss function.

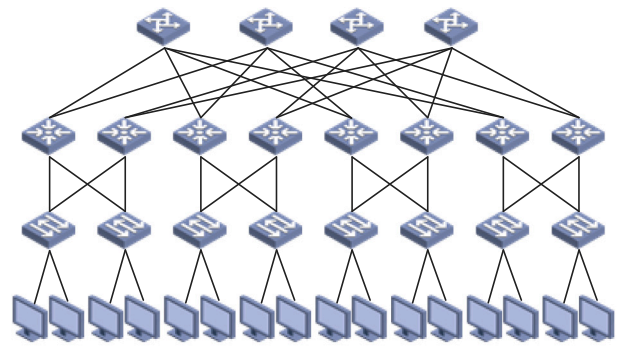

Fig. 3. The fat-tree topology.

5. Experiment and evaluation

5.1. Experimental setup

We conduct experiments on an Ubuntu 18.04 OS machine with Intel(R) Core(TM) i7-10700 2.90 GHz CPU and 32 GB RAM. To implement flexible INT, we use Mininet to create a virtual network environment. The BMv2 P4 software switch (P4lang, 2020) is used to realize the programmable environment to define the INT data packet header. In addition, Python's Scapy library is used to write scripts for probe packet generation and collection with a background traffic bandwidth of 2 Mbps, and the telemetry frequency is changed by adjusting the process sleep time. We adopt Python to develop the anomaly analysis module to process the network status information from telemetry. In order to evaluate the experimental performance, the fat-tree topology shown in Fig. 3 is selected as the experimental topology.

We use CAIDA (2020) traffic traces as background traffic in the above topology to generate normal traffic, and then construct different network anomalies through hping3 for injection. By sending UDP packets at high speed at intervals, hping3 can simulate micro-burst caused by instantaneous high traffic, and UDP flood can cause buffer overflow and packet loss, SYN flood attack can cause resource depletion of the target device and reduce the processing rate, and hping3 can also send a large number of ICMP requests, resulting in delay anomaly. In addition, when constructing background traffic, the link load can be controlled by sending packets through specified paths, which may also lead to ECMP load imbalance network anomaly. We simulated IP data flow in the simulation environment of the above network topology, and then captured the data flow in the Mininet simulation platform, in which we collected a total of 4251 normal flows and 1321 abnormal flows, and used INT technology to extract network state information and store it in the telemetry information matrix M.

In addition to the traffic data generated and telemetry in the above topology environment, we also use the NF-BoT-IoT-v2 dataset (Sarhan et al., 2022) to validate our proposed malicious intrusion detection model. This dataset is created by the University of Queensland, and it is a widely used and efficient research dataset. This dataset extracts 43 features from publicly available pcap files to describe network traffic, containing a total of 37,763,497 labeled network data flows, including 135,037 benign samples, and the dataset has four attack categories, namely DDoS, DoS, reconnaissance, and theft. We consider DDoS attacks as the object of malicious intrusion identification. The total number of DDoS attack data in the data set is 18,331,847. We keep all benign samples and randomly sample the same number of attack samples. The features containing IP addresses in the data set are converted into integer data through the IPv4Address function, and the rest of the features are numeric data, which can be stored in the information matrix M during data preprocessing, and then used as

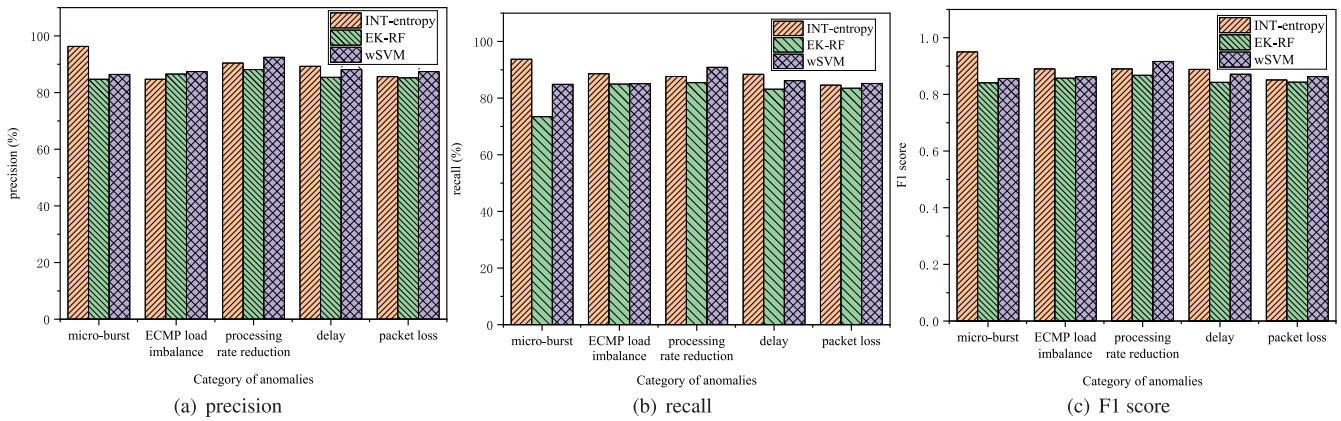

Fig. 4. Performance of different methods to identify five categories of network anomalies.

Table 3
Anomaly detection performance of different telemetry methods.

Method	Precision	Recall	F1 score
sFlow	0.7966	0.6701	0.701
5 Hz INT	0.8039	0.82	0.8122
10 Hz INT	0.8852	0.8643	0.8747
25 Hz INT	0.9125	0.8972	0.9048
50 Hz INT	0.9406	0.9284	0.9345
100 Hz INT	0.9621	0.9349	0.9483

the input of the AGCN model. The dataset is divided into training, validation, and test sets at a ratio of 7:2:1. The ratio of normal and abnormal flows is kept when the dataset is partitioned. When training our proposed AGCN malicious intrusion detection model, the learning rate is set to 0.01 and the training epoch is 200 times.

5.2. Performance evaluation

5.2.1. The effectiveness of INT

5.2.1.1. INT network telemetry parameters. In order to verify the effectiveness of using INT to collect network situation elements for anomaly detection, we design a comparison experiment of the influence of network state collection methods and telemetry parameters on anomaly detection. Firstly, the network anomaly events are injected into the current topology, and the network state information is collected by sFlow and INT with different telemetry frequencies respectively, and then it is applied to the network anomaly detection method based on information entropy. Finally, we calculate the detection precision, recall rate, and F1 score to compare the impact of different network telemetry methods on the network anomaly detection effect. The comparison results are shown in Table 3.

Compared with the collection method using INT, the effect of sFLow is not good, because sFlow is mainly used for the statistical analysis of network traffic, its collection information accuracy is limited, and the mean value is used to supplement the state information not collected. When the network device supports INT, it can obtain the detailed information of the packets in each node of the network, providing fine-grained flow information at the micro level. According to the experimental results in Table 3, high-frequency telemetry maintains better anomaly detection performance. When INT is used at 100 Hz, 97.21% precision, 93.49% recall rate, and 95.36% F1 value are achieved. Therefore, compared with the traditional network state acquisition method, the high-frequency INT can obtain the time series information and spatial correlation in the network state information, ensuring the fine granularity of network anomaly detection data, and achieve high-performance network anomaly detection.

5.2.1.2. Anomaly awareness performance. Network anomaly awareness needs to comprehensively consider its precision, recall rate, F1 score, and the latency of anomaly awareness. We explore the effectiveness of the anomaly detection method based on information entropy in INT (INT-entropy). The performance of the proposed method was compared with different anomaly detection methods, and the comparison methods were set as ensemble K-means++ and random forest (EK-RF) (Firdaus et al., 2020) and weighted SVM (wSVM) (Barona and Baburaj, 2023) for classification.

In this experiment, the telemetry frequency of INT-entropy is 100 Hz. In the comparison method, we use OpenFlow protocol to obtain network state information to realize anomaly detection, obtain anomaly awareness performance indicators and record the latency of anomaly awareness. Fig. 4 shows the performance of different methods in identifying five categories of network anomalies. Compared with the wSVM method, the performance of INT_entropy is slightly reduced in the processing rate reduction and packet loss two network anomalies, which are reduced by 2.03% and 1.79% respectively. However, INT focuses on fine-grained measurement and real-time telemetry of network states. Due to the fine-grained measurement of INT, INT-entropy performs best in identifying micro-burst anomalies, with precision, recall, and F1 score of 96.32%, 93.73%, and 0.9501, respectively, which are about 10% higher than those of the comparison methods.

The anomaly awareness latency refers to the time it takes for the network administrator to perceive the existence of a network anomaly. We record the timestamp when the abnormal packet flows out of the switch port and the timestamp when the network administrator receives the abnormal signal. The outgoing port timestamp of the data packet is t_1, and t_2 is the timestamp when the system detects the anomaly and sends an alert to the administrator, then the anomaly awareness latency is $t_2 - t_1$. In practical deployment scenarios, there may be additional latency errors, that is, the latency of transmitting the identified anomaly results to the network administrator. However, when actually deployed, the transmitted data packets only contain the anomaly result values, and the transmission latency of this content identified by different methods in the link is almost the same. Therefore, we can ignore this latency error. In the performance of anomaly awareness latency shown in Fig. 5, due to the real-time telemetry of INT-entropy, telemetry information can be directly embedded in the data packet transmission, and the design of information entropy in the edge layer can quickly detect network anomalies according to telemetry information. The comparison method implemented by OpenFlow protocol needs to realize traffic forwarding and processing in the communication and instruction transmission between the data plane and the control plane in SDN environment, so INT_entropy is significantly better than the comparison method in the awareness latency of five types of network anomalies, and the anomaly awareness latency of the comparison method is more than 860 ms. However, the average

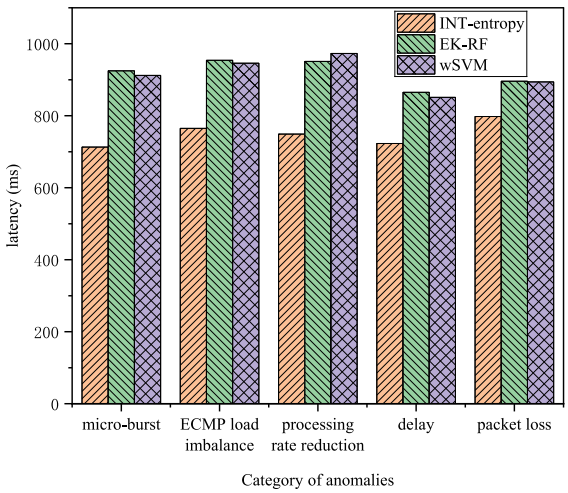

Fig. 5. Anomaly awareness latency of different methods.

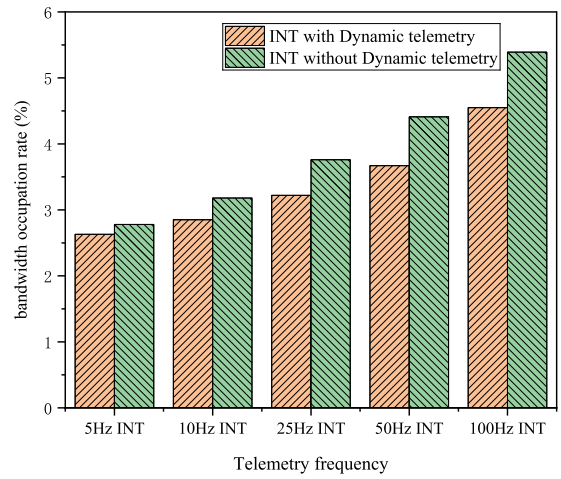

Fig. 6. Telemetry overhead comparison.

awareness latency of INT-entropy in five types of network anomalies is 750 ms, especially the awareness latency of micro-burst anomalies is 713 ms. When network anomalies are detected, it can provide abnormal warnings for network administrators in time.

In conclusion, compared with the other two anomaly detection methods, although the detection performance of INT anomaly detection based on information entropy is slightly worse in the anomaly category of processing rate reduction, it can quickly sense network anomalies according to historical information and real-time telemetry data, and its overall anomaly awareness latency has a significant advantage.

5.2.1.3. Telemetry overhead comparison. In order to verify whether the INT with dynamic telemetry mechanism can control the telemetry overhead more effectively, we evaluate the bandwidth occupancy caused by it at different telemetry frequencies in the network anomaly detection phase. It can be seen from the experimental results in Fig. 6 that under different telemetry frequency settings, compared with static telemetry, dynamic telemetry mechanism can control bandwidth occupancy more effectively. At the telemetry frequency of 5 Hz, due to the low telemetry frequency, the overall telemetry data is less, and there is a small difference in bandwidth overhead whether the dynamic telemetry mechanism is adopted. However, at the high frequency of 100 Hz telemetry, due to the high frequency of telemetry, the amount of telemetry data generation increases significantly, and its bandwidth occupation rate is 0.0539. The dynamic telemetry mechanism reduces redundant data collection by dynamically adjusting telemetry instructions according to network abnormal information, and its bandwidth occupation rate is 0.0455. It can be seen that the bandwidth occupation rate in this experimental example is reduced by 15.58% compared with the method without dynamic telemetry mechanism. In summary, the dynamic telemetry mechanism can adjust the telemetry instructions according to the actual situation of the network, effectively control the telemetry overhead, and improve the overall performance of the system.

5.2.2. Effectiveness of INT-AGCN

5.2.2.1. Performance of malicious intrusion detection. In order to better measure the performance of different models for malicious intrusion detection, this experiment compares the performance of the proposed INT-AGCN with GCN, 1D CNN (Altangerel et al., 2022), RNN (Nam et al., 2020), and CNN-LSTM (Zhao et al., 2021b). The performance indicators of all methods are shown in Table 4. The characteristics of DDoS attacks in malicious intrusions are forged by a large number of source IP addresses and port numbers, and it is easy to mistakenly identify micro-bursts in network anomalies as DDoS attacks, resulting in low accuracy of comparison methods. AGCN can fully detect malicious intrusions in anomalies by analyzing network abnormal information, and the detection accuracy can reach 98.69%.

Table 4
Comparison of malicious intrusion detection performance.

Method	Precision	Recall	F1 score
AGCN	0.9869	0.9801	0.9835
GCN	0.9653	0.9592	0.9622
1D CNN (Altangerel et al., 2022)	0.9327	0.9301	0.9314
RNN (Nam et al., 2020)	0.9125	0.9046	0.9085
CNN-LSTM (Zhao et al., 2021b)	0.9615	0.9579	0.9597

Since the constructed traffic trajectory graph contains the topological relationship of traffic, the overall evaluation performance index of AGCN and GCN methods is better than other comparison methods by learning the spatio-temporal characteristics of INT information. The recall rate and F1 score of the AGCN method are 98.01% and 98.35%, respectively. In contrast, the performance of the three types of benchmark methods, 1D CNN, RNN, and CNN-LSTM, is average. Compared with the above three types of benchmark methods, the F1 scores of the AGCN method are improved by 5.59%, 8.25%, and 2.48%, respectively, which effectively proves the effectiveness of the proposed AGCN method.

5.2.2.2. Analysis of convergence. The malicious intrusion in the network anomaly needs to be further determined. Different from the rapid identification of network anomalies at the edge layer, the identification of malicious intrusion requires accurate detection through model training in the cloud layer. In order to verify the convergence of the proposed INT-AGCN method for malicious intrusion detection, we compare the training iteration process of the method using GCN directly and the AGCN method with abnormal information. As shown in Fig. 7, GCN method and AGCN method have large changes in accuracy and loss in the early epoch, but AGCN combines the abnormal information detected based on information entropy for learning, so its model can further converge rapidly in a small number of epochs, and soon tends to be stable at a high accuracy. When the GCN model is trained to 70 epochs, the accuracy of the training set reaches 96.98%, and remains stable at about 98.2% after 100 epochs, while the accuracy of the training set of the AGCN model has reached 98.5% after 75 epochs. In addition, the loss value of the AGCN method decreases rapidly at the beginning and tends to be stable by 50 epochs, while the GCN method maintains stability after 100 epochs. In summary, the AGCN method with anomaly detection information proposed in this paper has faster training convergence speed, and has higher accuracy than the GCN method after convergence, which can quickly complete the identification of malicious intrusions.

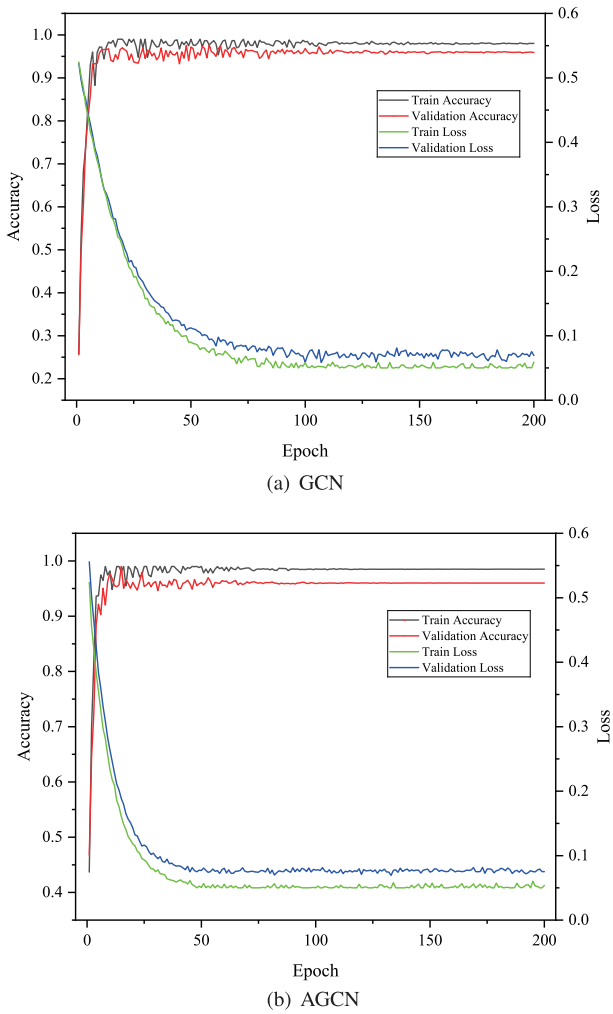

Fig. 7. The change curve of accuracy and loss value during training.

6. Conclusions

The rapid perception of security events in a cloud–edge collaborative environment can make the cloud–edge collaborative system more secure. However, the delayed and inaccurate network telemetry information obtained from the terminal devices at the edge of the network has the defects of poor flexibility and a single type of measurement data, which makes it difficult to accurately and specifically perceive network security situation. Besides, a large number of fine-grained telemetry data will cause huge telemetry overhead. In this paper, INT is used to collect fine-grained network state information, and we propose a novel anomaly-based intrusion detection system for cloud–edge collaborative networks. Firstly, INT was used to extract the network situation elements, and then the information entropy method was used to quickly identify five kinds of network anomalies. In addition, we designed a dynamic telemetry architecture, and then used AGCN to aggregate abnormal information in the cloud layer to detect whether there was malicious intrusion in network anomalies, so that network managers could effectively grasp the network operation.

In the future, it is necessary to realize the integration of anomaly awareness and network configuration management system. Once the anomaly is found, it automatically triggers the anomaly repair operation or sends accurate fault reports to the network administrator. For example, if abnormal traffic is detected, the system automatically adjusts the network configuration parameters of the network device to isolate the abnormal device and prevent the threat from spreading. In addition, the system requires continuous monitoring and learning network behavior mode, optimizing the detection model and the threshold setting, so as to adapt to the change of network environment and emerging threats.

CRediT authorship contribution statement

Jinchuan Pei: Writing – original draft, Software, Methodology, Conceptualization. **Yuxiang Hu:** Supervision, Resources, Funding acquisition. **Le Tian:** Supervision, Project administration. **Xinglong Pei:** Validation, Investigation, Formal analysis. **Zihao Wang:** Visualization, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Key R&D Program of China under Grant 2023YFB2903902; in part by the Science and Technology Innovation leading Talents Subsidy Project of Central Plains under Grant 244200510038; and in part by the Program of Songshan Laboratory (Included in the management of Major Science and Technology Program of Henan Province) under Grant 221100210900-02.

Data availability

The data that has been used is confidential.

References

Abdallah, M., An Le Khac, N., Jahromi, H., Delia Jurcut, A., 2021. A hybrid CNN-LSTM based approach for anomaly detection systems in SDNs. In: Proceedings of the 16th International Conference on Availability, Reliability and Security. pp. 1–7.

Altangerel, G., Tejfel, M., Tsogbaatar, E., 2022. A 1D CNN-based model for IoT anomaly detection using INT data. In: 2022 IEEE 16th International Scientific Conference on Informatics (Informatics). IEEE, pp. 106–113.

Barona, R., Baburaj, E., 2023. An efficient DDoS attack detection and categorization using adolescent identity search-based weighted SVM model. Peer-To-Peer Netw. Appl. 16 (2), 1227–1241.

CAIDA, 2020. The CAIDA UCSD anonymized internet traces. https://www.caida.org/data/passive/passive_dataset.xml. (Accessed 7 September 2024).

Cao, Y., Jiang, H., Deng, Y., Wu, J., Zhou, P., Luo, W., 2021. Detecting and mitigating DDoS attacks in SDN using spatial-temporal graph convolutional network. IEEE Trans. Dependable Secur. Comput. 19 (6), 3855–3872.

Case, J.D., Fedor, M., Schoffstall, M.L., Davin, J., 1988. Simple Network Management Protocol. Tech. Rep..

Changhoon, K., Parag, B., Ed, D., Hugh, H., Anoop, G., Dan, D., Mukesh, H., Bruce, D., 2020. In-band Network Telemetry (INT). https://github.com/p4lang/p4-applications/tree/master/docs/INT_v2_1.pdf. (Accessed 7 September 2024).

Duan, Y., Li, C., Bai, G., Chen, G., Zhou, F., Chen, J., Gao, Z., Zhang, C., 2023. MFGAD-INT: in-band network telemetry data-driven anomaly detection using multi-feature fusion graph deep learning. J. Cloud Comput. 12 (1), 126.

Fernandez, D., Lorenzo, H., Novoa, F.J., Cacheda, F., Carneiro, V., 2017. Tools for managing network traffic flows: A comparative analysis. In: 2017 IEEE 16th International Symposium on Network Computing and Applications. NCA, IEEE, pp. 1–5.

Firdaus, D., Munadi, R., Purwanto, Y., 2020. Ddos attack detection in software defined network using ensemble k-means++ and random forest. In: 2020 3rd International Seminar on Research of Information Technology and Intelligent Systems. ISRITI, IEEE, pp. 164–169.

Guo, C., Yuan, L., Xiang, D., Dang, Y., Huang, R., Maltz, D., Liu, Z., Wang, V., Pang, B., Chen, H., et al., 2015. Pingmesh: A large-scale system for data center network latency measurement and analysis. In: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication. pp. 139–152.

Handigol, N., Heller, B., Jeyakumar, V., Mazières, D., McKeown, N., 2014. I know what your packet did last hop: Using packet histories to troubleshoot networks. In: 11th USENIX Symposium on Networked Systems Design and Implementation. NSDI 14, pp. 71–85.

Hnamte, V., Hussain, J., 2023. An efficient DDoS attack detection mechanism in SDN environment. Int. J. Inf. Technol. 15 (5), 2623–2636.
Jiang, B., He, Q., Zhai, Z., Su, H., 2023. Anomaly detection and access control for cloud-edge collaboration networks. Intell. Autom. Soft Comput. 37 (2).
Li, Q., Huang, H., Li, R., Lv, J., Yuan, Z., Ma, L., Han, Y., Jiang, Y., 2023. A comprehensive survey on DDoS defense systems: New trends and challenges. Comput. Netw. 109895.
Liu, C., He, L., Xiong, G., Cao, Z., Li, Z., 2019. Fs-net: A flow sequence network for encrypted traffic classification. In: IEEE INFOCOM 2019-IEEE Conference on Computer Communications. IEEE, pp. 1171–1179.
Liu, K., Jiang, Z., Zhang, J., Guo, S., Zhang, X., Bai, Y., Dong, Y., Luo, F., Zhang, Z., Wang, L., et al., 2024. R-pingmesh: A service-aware RoCE network monitoring and diagnostic system. In: Proceedings of the ACM SIGCOMM 2024 Conference. pp. 554–567.
Nam, S., Lim, J., Yoo, J.-H., Hong, J.W.-K., 2020. Network anomaly detection based on in-band network telemetry with RNN. In: 2020 IEEE International Conference on Consumer Electronics-Asia. ICCE-Asia, IEEE, pp. 1–4.
P4lang, 2020. Behavioral model version 2 (BMv2). https://github.com/p4lang/behavioral-model. (Accessed 7 September 2024).
Qu, L., Wang, Y., Yang, J., Zhao, M., 2023. A heterogeneous network structure publishing security framework based on cloud-edge collaboration. Comput. Netw. 234, 109947.
Rasley, J., Stephens, B., Dixon, C., Rozner, E., Felter, W., Agarwal, K., Carter, J., Fonseca, R., 2014. Planck: Millisecond-scale monitoring and control for commodity networks. ACM SIGCOMM Comput. Commun. Rev. 44 (4), 407–418.
Sacco, A., Angi, A., Esposito, F., Marchetto, G., 2023. HINT: Supporting congestion control decisions with P4-driven in-band network telemetry. In: 2023 IEEE 24th International Conference on High Performance Switching and Routing. HPSR, IEEE, pp. 83–88.
Sarhan, M., Layeghy, S., Portmann, M., 2022. Towards a standard feature set for network intrusion detection system datasets. Mob. Netw. Appl. 27, 357–370.
Singh, H., Huang, C., Sicard-Gagne, M., Shami, G., Lyonnais, M., Fedorov, D., Wilson, R., 2019. INT-SDN: Evaluation of various P4 parameters using optical telemetry having reconfigurable data plane on 40 Gbps line rate. In: 2019 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing. PACRIM, IEEE, pp. 1–6.
Tan, C., Jin, Z., Guo, C., Zhang, T., Wu, H., Deng, K., Bi, D., Xiang, D., 2019. {NetBouncer}: Active device and link failure localization in data center networks. In: 16th USENIX Symposium on Networked Systems Design and Implementation. NSDI 19, pp. 599–614.
Tilmans, O., Bühler, T., Poese, I., Vissicchio, S., Vanbever, L., 2018. Stroboscope: Declarative network monitoring on a budget. In: 15th USENIX Symposium on Networked Systems Design and Implementation. NSDI 18, pp. 467–482.
Ujjan, R.M.A., Pervez, Z., Dahal, K., Bashir, A.K., Mumtaz, R., González, J., 2020. Towards sFlow and adaptive polling sampling for deep learning based DDoS detection in SDN. Future Gener. Comput. Syst. 111, 763–779.
Wang, B., Chen, H., Chen, P., He, Z., Yu, G., 2023. MARS: Fault localization in programmable networking systems with low-cost in-band network telemetry. In: Proceedings of the 52nd International Conference on Parallel Processing. pp. 347–357.
Wang, W., Zhu, M., Zeng, X., Ye, X., Sheng, Y., 2017. Malware traffic classification using convolutional neural network for representation learning. In: 2017 International Conference on Information Networking. ICOIN, IEEE, pp. 712–717.
Yang, F., Quan, W., Cheng, N., Xu, Z., Zhang, X., Gao, D., 2020. Fast-INT: Light-weight and efficient in-band network telemetry in programmable data plane. In: 2020 IEEE 92nd Vehicular Technology Conference. VTC2020-Fall, IEEE, pp. 1–5.
Yao, H., Mai, T., Xu, X., Zhang, P., Li, M., Liu, Y., 2018. NetworkAI: An intelligent network architecture for self-learning control strategies in software defined networks. IEEE Internet Things J. 5 (6), 4319–4327.
Zhan, P., Qin, G., Qian, X., Wang, X., Ren, J., Zhuang, Z., Xu, S., 2023. Deep reinforcement learning based fast anomaly detection and localization for programmable networks. In: ICC 2023-IEEE International Conference on Communications. IEEE, pp. 4866–4872.
Zhang, Y., Pan, T., Zheng, Y., Gao, M., Wang, H., Huang, T., Liu, Y., 2021. Automating rapid network anomaly detection with in-band network telemetry. IEEE Netw. Lett. 4 (1), 39–42.
Zhang, S., Tong, H., Xu, J., Maciejewski, R., 2019. Graph convolutional networks: a comprehensive review. Comput. Soc. Netw. 6 (1), 1–23.
Zhao, Y., Cheng, G., Duan, Y., Gu, Z., Zhou, Y., Tang, L., 2021a. Secure IoT edge: Threat situation awareness based on network traffic. Comput. Netw. 201, 108525.
Zhao, R., Deng, X., Wang, Y., Chen, L., Liu, M., Xue, Z., Wang, Y., 2022. Flow sequence-based anonymity network traffic identification with residual graph convolutional networks. In: 2022 IEEE/ACM 30th International Symposium on Quality of Service. IWQoS, IEEE, pp. 1–10.
Zhao, Z., Li, Z., Li, F., Liu, Y., 2021b. CNN-LSTM based traffic prediction using spatial-temporal features. In: Journal of Physics: Conference Series. Vol. 2037, IOP Publishing, 012065.
Zhu, Y., Kang, N., Cao, J., Greenberg, A., Lu, G., Mahajan, R., Maltz, D., Yuan, L., Zhang, M., Zhao, B.Y., et al., 2015. Packet-level telemetry in large datacenter networks. In: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication. pp. 479–491.

Jinchuan Pei received the B.S. degree from Taiyuan University of Technology, Taiyuan, China, in 2020. He received the M.S. degree from the National Digital Switching System Engineering & Technological R&D Center (NDSC), Information Engineering University, China., in 2023. He is currently pursuing the Ph.D. degree from the National Digital Switching System Engineering & Technological R&D Center (NDSC), Information Engineering University, China. His current research interests include traffic engineering, network security, and machine learning.

Yuxiang Hu received the Ph.D. degree from National Digital Switching System Engineering & Technological R&D Center (NDSC), Information Engineering University, China, in 2011. He is currently a Professor and doctoral advisor. His research areas are next generation network architecture and network security.

Le Tian is an associate professor at Information Engineering University and National Digital Switching System Engineering and Technological R&D Center (NDSC), China. He received his Ph.D. degree in Computer Science from the University of Antwerp in the Internet technology and Data science LAB (IDLAB), Belgium, in 2019. His research focuses on Internet architecture, software defined networking, P4 programmable networks and IEEE 802.11 WLAN.

Xinglong Pei received the B.S. degree from Zhengzhou University of Light Industry, Zhengzhou, China, in 2019. He received the M.S. degree from Chongqing University of Technology, Chongqing, China, in 2022. He is currently pursuing the Ph.D. degree from the National Digital Switching System Engineering & Technological R&D Center (NDSC), Information Engineering University, China. His current research interests include traffic engineering, network architecture, and machine learning.

Zihao Wang received the B.S. degree from Zhengzhou University, Zhengzhou, China, in 2020. He received the M.S. degree from Zhengzhou University, Zhengzhou, China, in 2023. He is currently pursuing the Ph.D. degree from the National Digital Switching System Engineering & Technological R&D Center (NDSC), Information Engineering University, China. His current research interests include network security, machine learning, zero trust network.